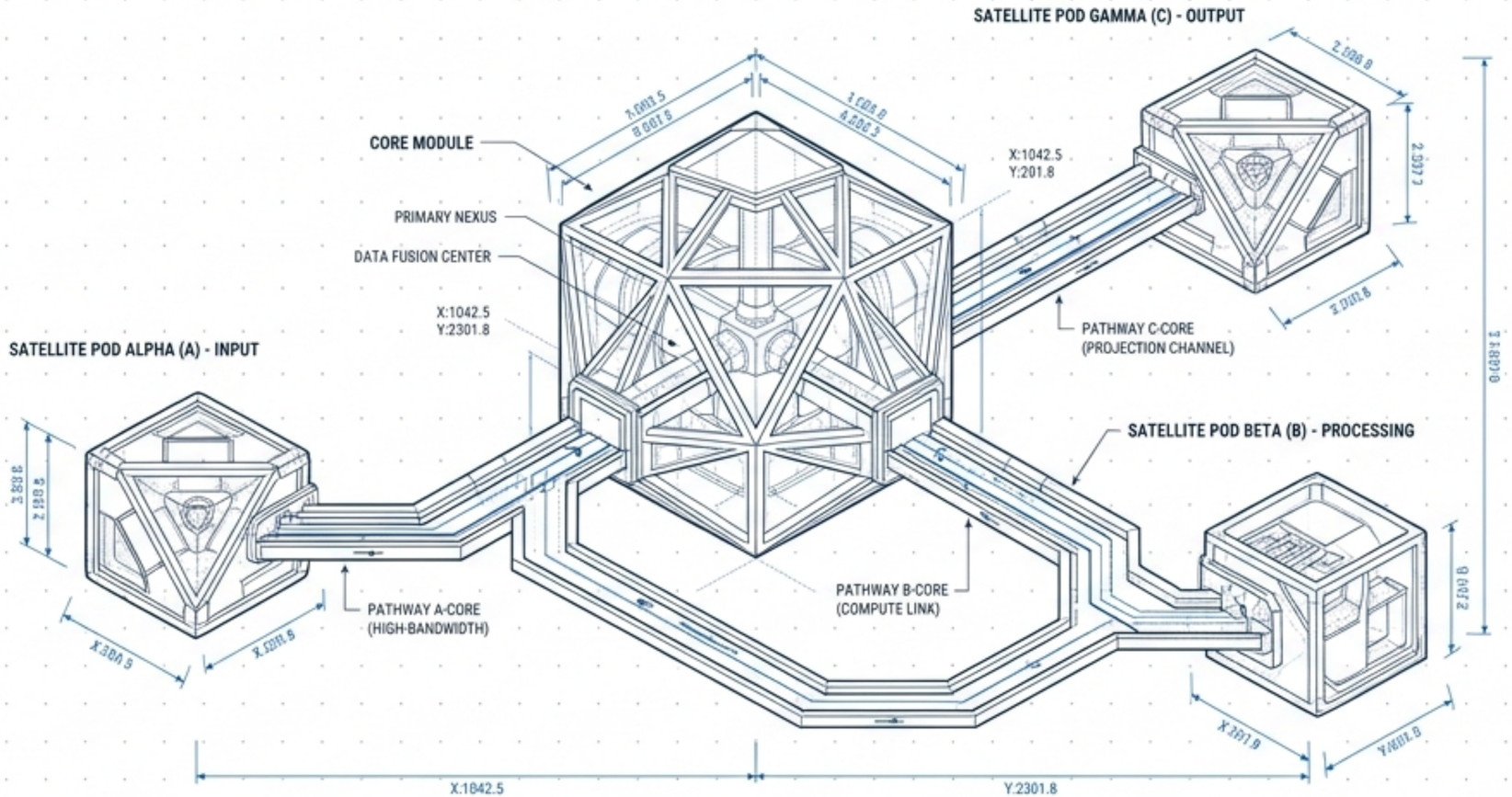
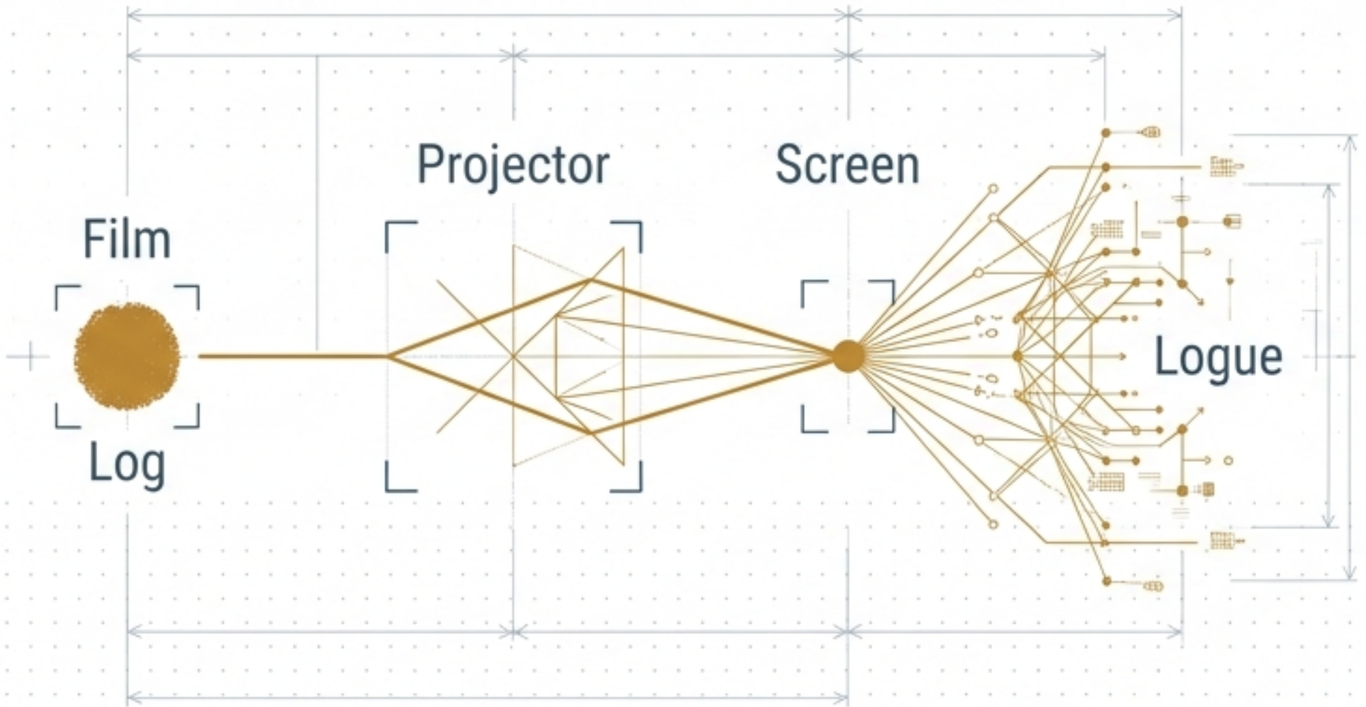


魂のホログラフィ母艦 — LogからLogueへ

ryubostyle/Templates · Base-Class Architecture Specification · v1.0

設計思想：Filmに魂を刻み、Projectorで意味を解き、Screenに世界を結ぶ。

Gemini Notebook



記録から対話へ、知識から意味へ

このリポジトリは、単なるMarkdownテンプレートの集合ではありません。共通の思想・データ構造・変換規則を持つ「基底クラスと派生クラスの体系」です。

記録された事実（Log）を、対話によって意味づける物語（Logue）へ変換すること。それがこのアーキテクチャの核心です。

Gemini Notebook

5つの設計の不変原則



Filmは宣言する。

YAMLに構造と意味を記録。実行ロジックは持たない。

Projectorは解釈する。

共通処理は基底に。子クラスは固有の変換規則のみ。

Screenは表現する。

出力形式と世界観を分離。同一データから複数表現を生成。

Logは事実を保持する。

原記録と、後付けの解釈・物語を混同しない。

Logueは対話から生まれる。

原記録を上書きせず、出典と解釈を区別する。

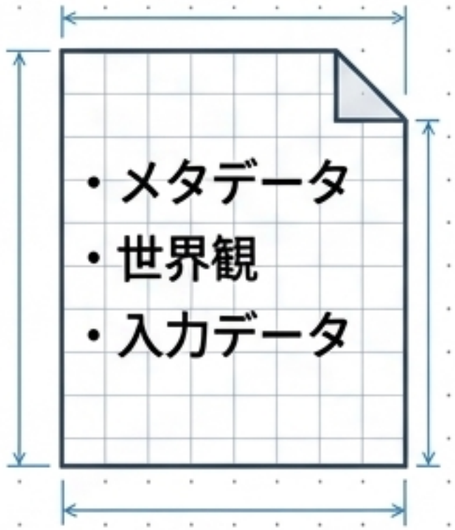
Gemini Notebook

基本アーキテクチャ：3段階の変換パイプライン

Phase 1 — Film

宣言的な仕様書

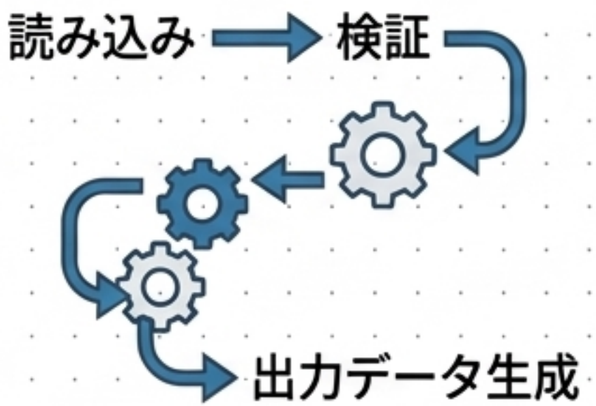
Stack: YAML



Phase 2 — Projector

共通変換エンジン

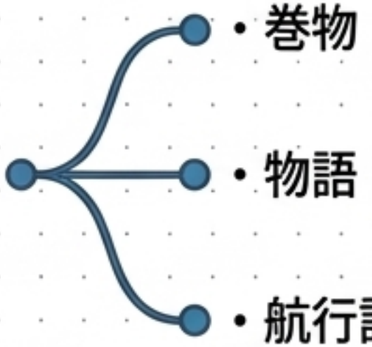
Stack: Python (BaseProjector)



Phase 3 — Screen

ホログラフィックな表現

Stack: Markdown



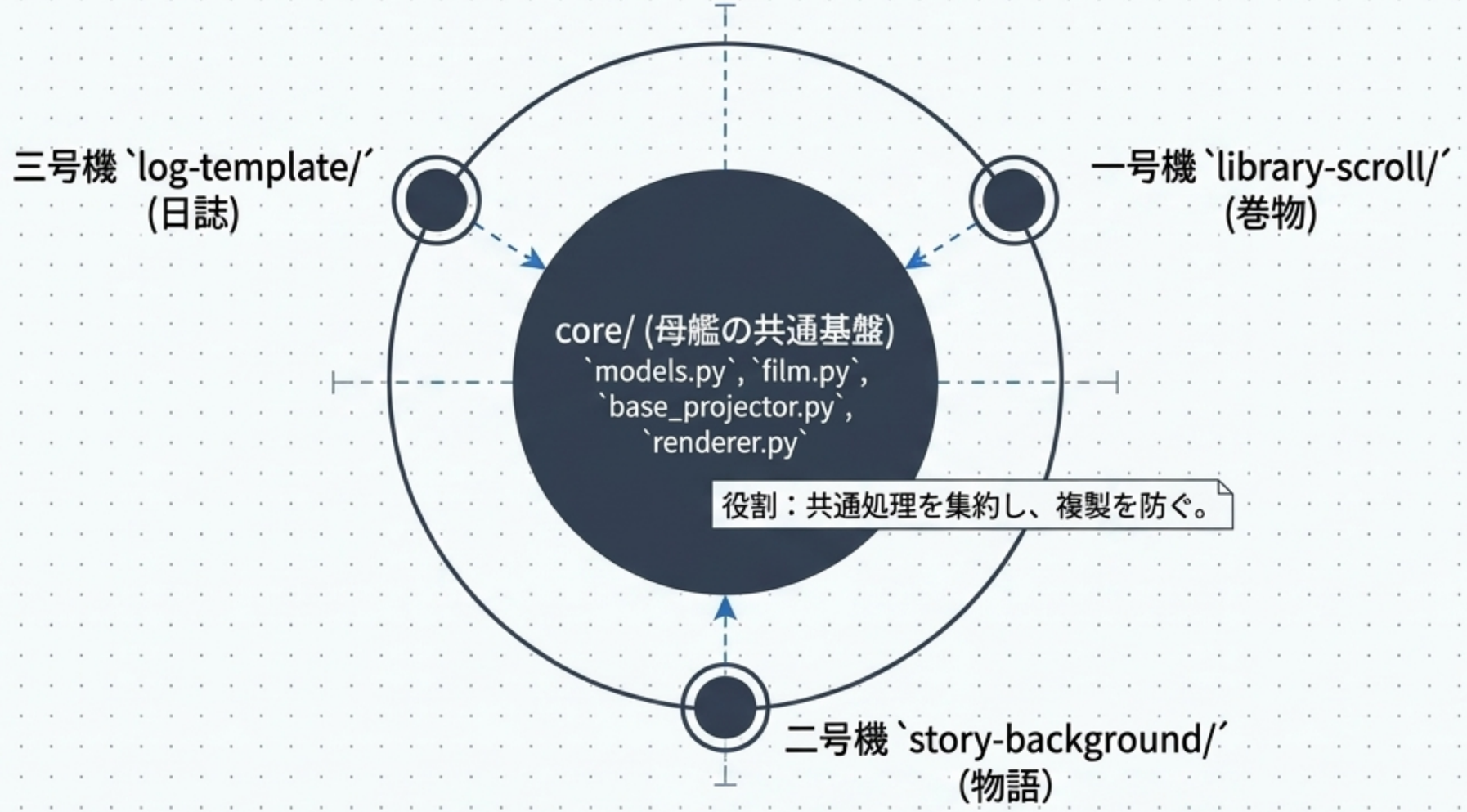
・巻物
・物語
・航行記録

(将来はWeb・音声)

単なるファイル変換ではなく、事実と意図を保持したまま異なる目的に展開する共通パイプライン。

Gemini Notebook

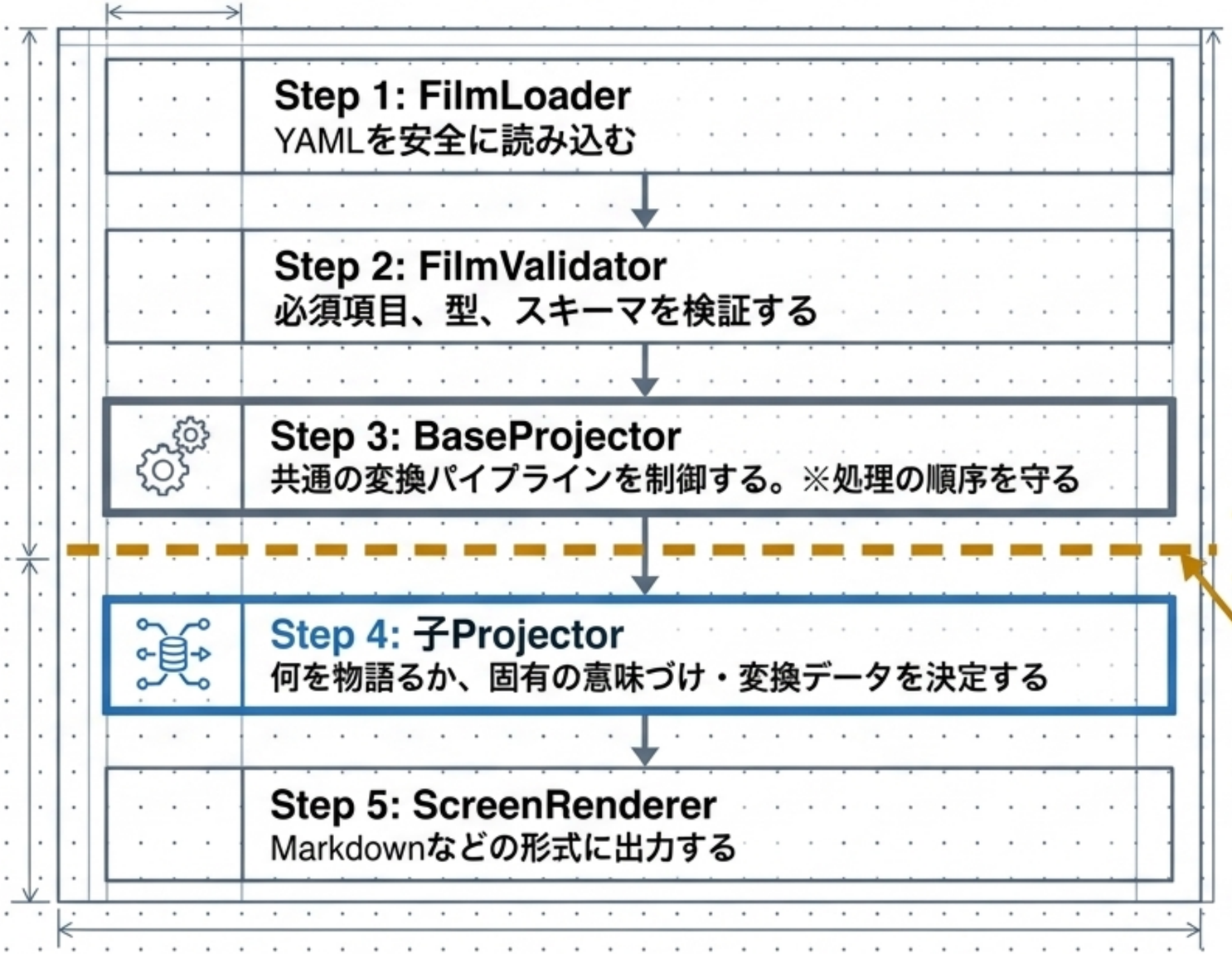
母艦と子機の同心円モデル：リポジトリ構成



3つの子ディレクトリは独立したテンプレートでありながら、母艦の共通基盤を共有する。

Gemini Notebook

基底クラスが定める「共通契約」



Key Takeaway:
基底クラスが決定するのは「処理の順序と契約」のみ。世界観の決定はすべて子クラスに委ねられる。

Gemini Notebook

BaseProjectorの実装骨格とインターフェース

BaseProjector Class Definition (Abstract)

```python
class BaseProjector(ABC):
    def from_yaml(cls, path: Path) -> 'BaseProjector': ...
    @abstractmethod
    def transform(self) -> dict[str, Any]: ...
    def project(self) -> Projection: ...
```

`transform(): 子クラス固有の変換を実行する契約。

`project(): 全テンプレート共通の投射パイプライン。

Usage Example (Implementation)

```python
projector = LibraryScrollProjector.from_yaml(...)
projection = projector.project()
```

Insight: 将来4機目、5機目を追加しても、母艦の呼び出しパイプライン（インターフェース）は一切変更する必要がない。

Gemini Notebook

Film：共通仕様と固有拡張の境界線

The Blueprint Document

Common Specification (母艦の管轄)

`metadata` (project_id, template_id)
`philosophy`
`input`
`projection`
`screen`

Specific Extension (子機の管轄)

`extensions: library_scroll:
(archive_title, organization, include_toc)

Validation Rules

- ☑ ・PyYAMLの safe_load() のみ使用 (危険なオブジェクト生成を排除)。
- ☑ ・未知の必須設定や不正な項目は無視せずエラーとする。
- ☑ ・extensions 内は template_id に対応する固有JSONスキーマで厳格に検証。

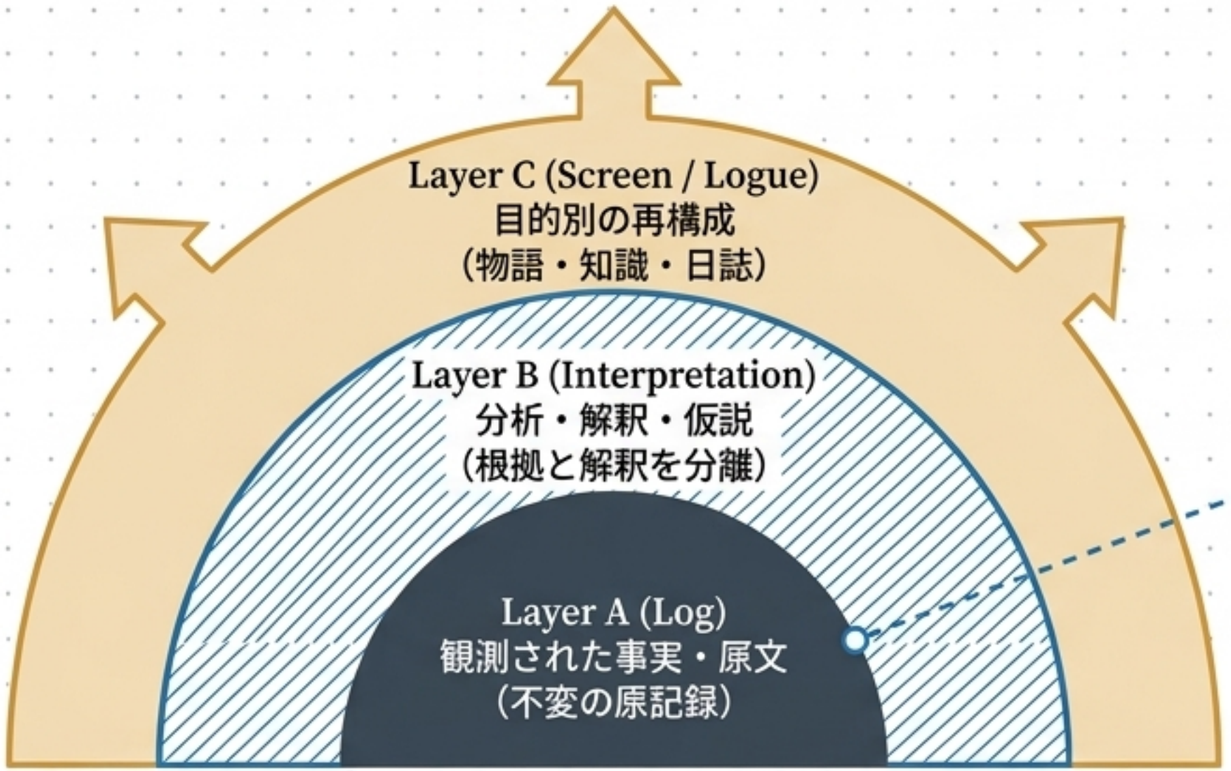
Gemini Notebook

3機の探査機：それぞれの役割と固有原則

	一号機 (Scroll)	二号機 (Story)	三号機 (Log)
役割 (Target)	知識を編む	経験を物語にする	変化を記録する
入力 (Input)	アーカイブ、概念	出来事、振り返り	タイムスタンプ、観測値
変換 (Transformation)	分類、関連づけ、目次生成	時系列整理、転換点抽出	時間経過計算、状態集計
出力 (Output)	巻物型Markdown、索引	背景物語、年表	航行日誌、統計サマリー
固有原則 (Unique Principle)	出典と解釈を分ける	実際の出来事と創作演出を区別する	原記録を保持し計算値と区別する

Gemini Notebook

LogからLogueへの3層データ地層図

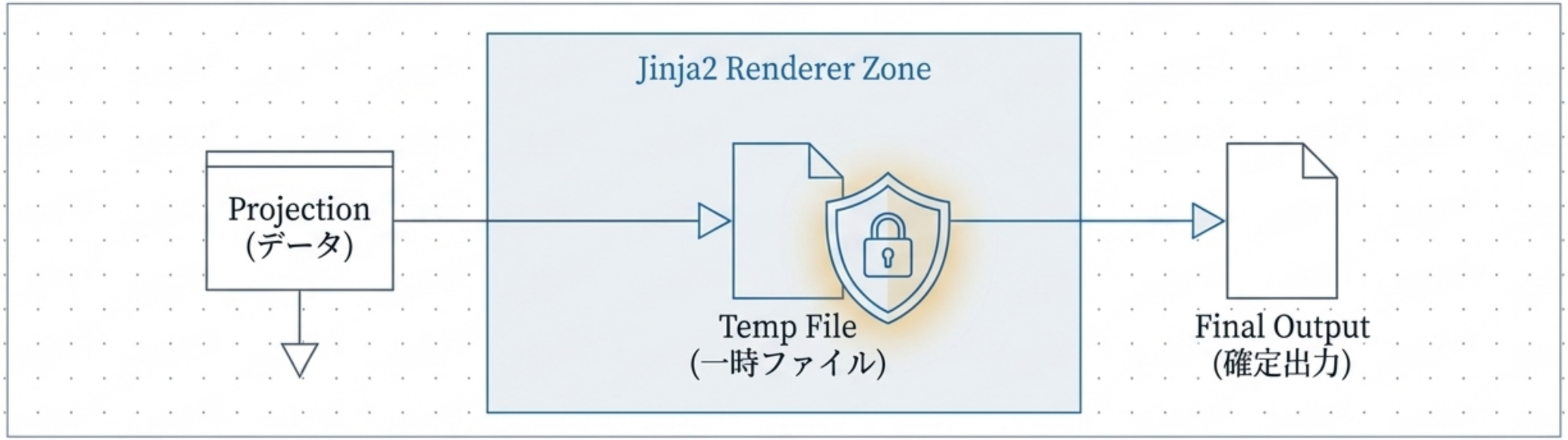


The 2FA Case Study

「二段階認証で苦労した」という単一の事実（Layer A）から、三号機は「時系列ログ」を、一号機は「認証技術の知識」を、二号機は「立ち上げの神話的物語」を生成する（Layer C）。事実は決して書き換わらない。

Gemini Notebook

Screenレンダラーと安全なファイル生成



Rule 1: Projectorはファイルへ直接書き込まない。レンダラーにデータを渡すのみ。

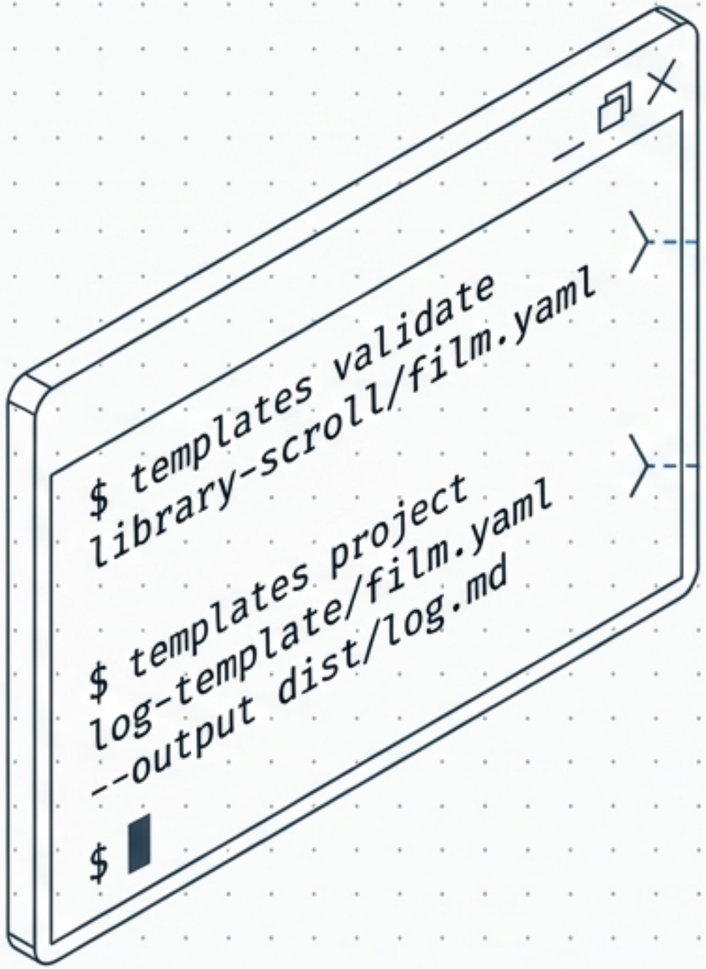
Rule 2: 許可されたディレクトリ内へのみ出力を制限する。

Rule 3: 一時ファイル（Temp）へ書き込み、正常生成が確認された場合のみ確定する。

Rule 4: 冪等性（Idempotency）。同じFilmと設定からは、常に同じ出力が再生成される。

Gemini Notebook

コマンドライン実行インターフェース (CLI)



YAMLの安全な読み込みと検証

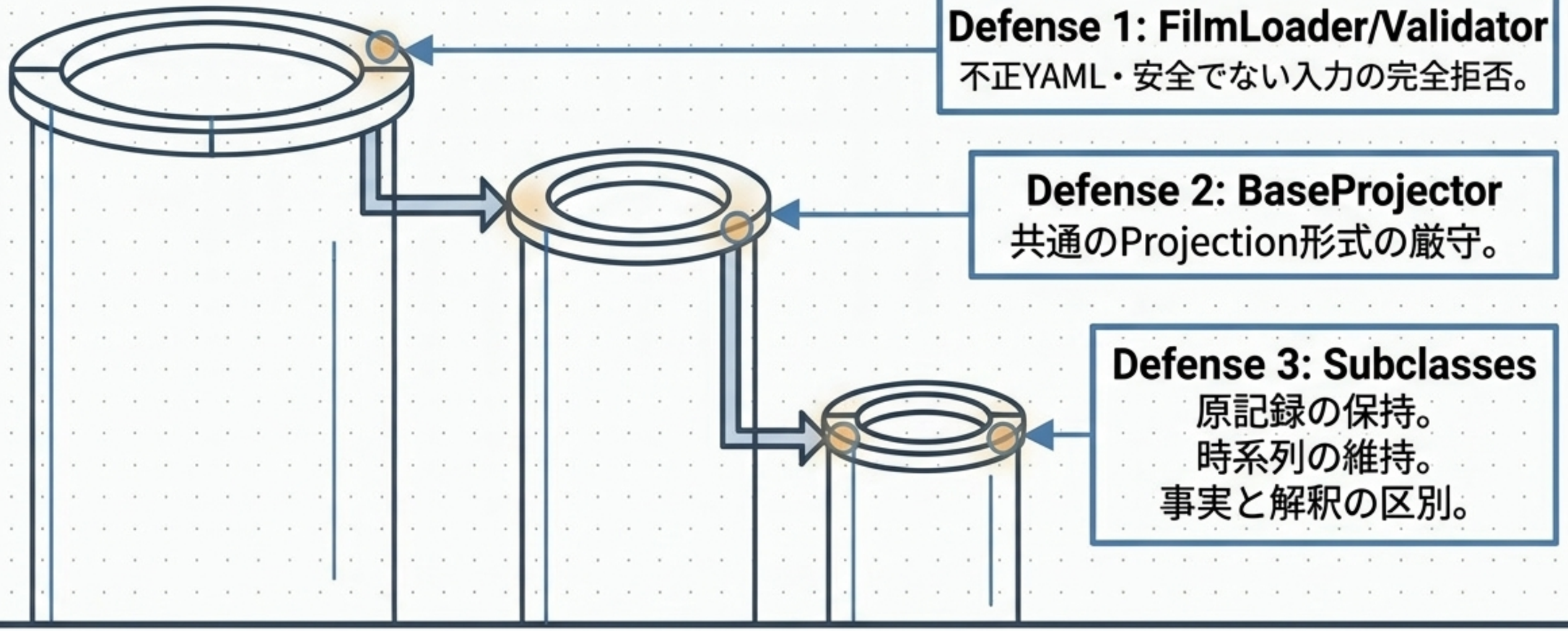
2. `template_id` によるProjectorの特定

3. `transform()` によるProjection生成

4. レンダラー経由でのScreen確定と完了報告

Gemini Notebook

品質保証：共通契約の防衛線



Defense 1: FilmLoader/Validator
不正YAML・安全でない入力の完全拒否。

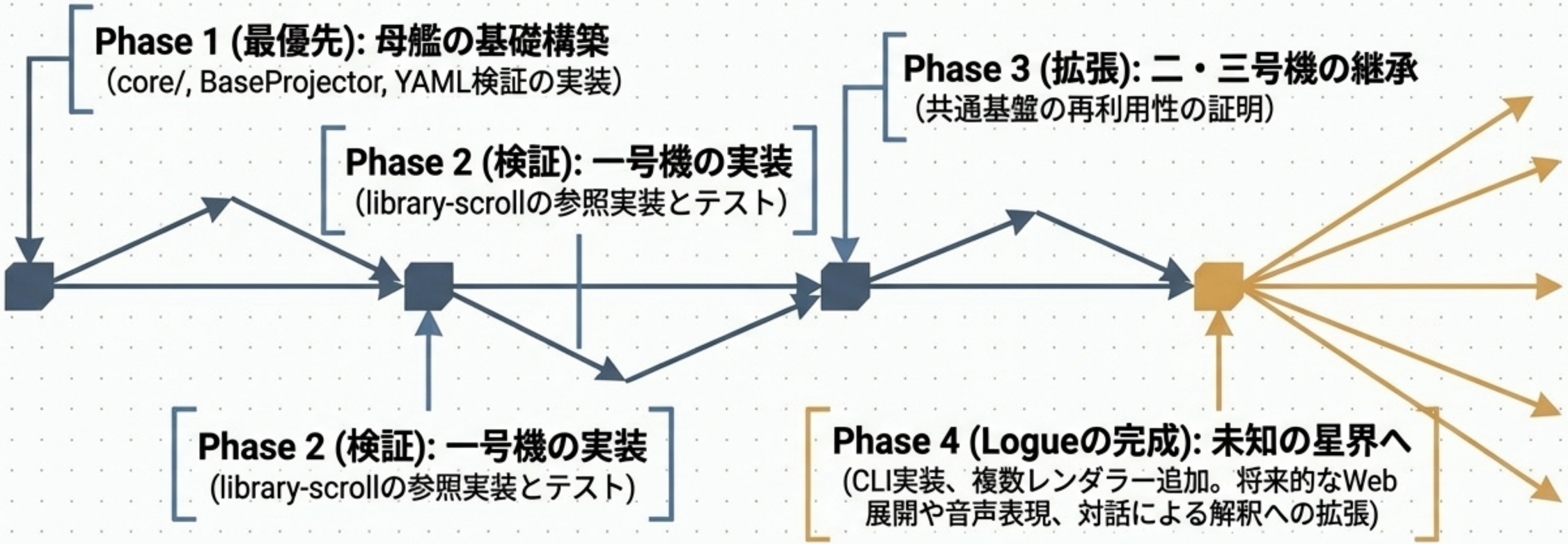
Defense 2: BaseProjector
共通のProjection形式の厳守。

Defense 3: Subclasses
原記録の保持。
時系列の維持。
事実と解釈の区別。

Integration (End-to-End)
FilmからScreenまで一貫処理。誤った`template_id`での即時フェイルセーフ。

Gemini Notebook

進化の軌跡：Phase 1から未知の星界へ



母艦の完成は終着点ではない。
4機目、5機目が同じDNAを持って飛び立つための「共通の出発点」である。

Gemini Notebook